

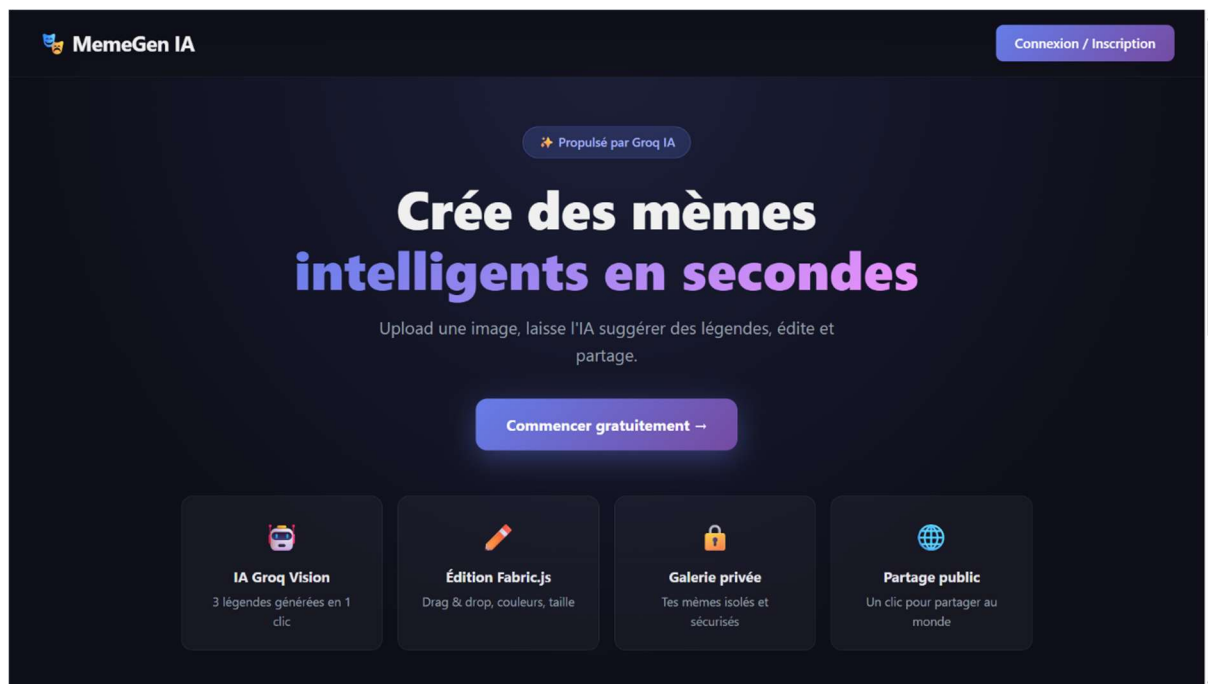


**Projet d'admission Master of Science — Intelligence Artificielle 4e année —  
Paris — Septembre 2026**

## MemeGen IA : Générateur de mèmes intelligent

Présenté par : Yves ANAGUE

Contact : [yvesanague06@gmail.com](mailto:yvesanague06@gmail.com)



Juin 2026

## **Présentation du projet**

Dans le cadre de ma candidature au programme Master of Science en Intelligence Artificielle de SUPINFO Paris, j'ai développé MemeGen IA, une application web complète de génération de mèmes assistée par intelligence artificielle.

L'application est accessible en ligne à l'adresse suivante : <https://memegen-supinfo.vercel.app>

Le code source complet est disponible sur GitHub : <https://github.com/Yves-ANAGUE/memegen-supinfo>

### **Ce que fait l'application :**

MemeGen IA permet à un utilisateur de créer un compte, d'uploader une photo depuis son ordinateur, de la modifier en ajoutant du texte directement sur l'image, puis de sauvegarder son mème dans une galerie personnelle ou de le publier pour que tout le monde puisse le voir. En un clic, l'intelligence artificielle analyse l'image et propose trois légendes humoristiques adaptées au contenu visuel.

Toutes les données sont sécurisées : chaque utilisateur ne peut accéder qu'à ses propres créations, même si techniquement il tente d'accéder à celles d'un autre.

## **Fonctionnalités réalisées**

Le projet répond à l'ensemble des exigences du sujet et va au-delà sur plusieurs points.

### **Téléchargement d'images**

L'utilisateur peut charger une image depuis son ordinateur aux formats JPEG, PNG ou WebP. L'application vérifie automatiquement que le fichier ne dépasse pas 5 Mo et que le format est supporté avant de l'afficher sur le canvas. Si l'image est trop grande, elle est redimensionnée automatiquement sans perte de qualité visible.

### **Édition interactive en temps réel**

L'éditeur est basé sur la librairie Fabric.js, qui transforme un simple canvas HTML en surface d'édition avancée. L'utilisateur peut ajouter autant de blocs de texte qu'il le souhaite, les déplacer à la souris, les redimensionner en tirant les poignées, modifier leur couleur et leur taille, et les supprimer avec la touche Suppr. Chaque modification est visible instantanément, sans délai ni rechargement.

### **Suggestions de légendes par intelligence artificielle**

En cliquant sur le bouton "Suggérer des légendes", l'image est envoyée à l'API Groq qui fait tourner le modèle Llama 4 Scout, un modèle de vision multimodale capable d'analyser le contenu d'une image. Le modèle retourne trois propositions de légendes humoristiques adaptées à l'image. Un clic sur une suggestion l'ajoute directement sur le canvas, prête à être repositionnée.

### **Téléchargement et partage**

L'utilisateur peut télécharger son même final en PNG d'un simple clic. Il peut également le partager via une fenêtre qui propose six réseaux sociaux : Twitter/X, Facebook, WhatsApp, Telegram, Reddit et LinkedIn. Sur mobile, la fonctionnalité de partage natif du navigateur est activée, permettant d'envoyer le fichier directement aux applications installées.

### **Galerie privée et galerie publique**

Chaque utilisateur dispose d'un espace personnel où toutes ses créations sont stockées. Il peut choisir de rendre un même public, ce qui le fait apparaître dans la galerie visible par tous les visiteurs de l'application. Depuis la galerie, il est possible de rouvrir un même dans l'éditeur pour le modifier, de le supprimer, ou de le télécharger directement.

### **Authentification sécurisée**

L'accès à l'éditeur et à la galerie personnelle est réservé aux utilisateurs connectés. L'inscription et la connexion sont gérées par Supabase Auth, avec un token JWT renouvelé automatiquement en arrière-plan.

## **Choix techniques et architecture**

### **Organisation du projet**

L'application est découpée en deux parties indépendantes : un frontend en React déployé sur Vercel, et un backend en Node.js déployé sur Railway. La base de données, le stockage des fichiers et l'authentification sont hébergés sur Supabase.

Cette séparation a une raison de sécurité précise : la clé API Groq, qui donne accès au service d'intelligence artificielle, ne doit jamais se trouver dans le code qui s'exécute dans le navigateur. Elle reste uniquement sur le serveur backend, invisible pour l'utilisateur final.

### **Pourquoi Fabric.js pour l'éditeur**

Fabric.js est la librairie la plus mature disponible pour manipuler des objets sur un canvas HTML. Elle gère nativement le drag and drop, le redimensionnement avec poignées, l'édition de texte en double-cliquant, et la sérialisation de l'état complet du canvas. Les alternatives comme Konva.js ont une API moins intuitive pour la gestion du texte éditable.

### **Pourquoi Supabase pour la sécurité des données**

Supabase propose une fonctionnalité appelée Row Level Security, qui permet de définir directement dans la base de données des règles d'accès par utilisateur. Concrètement, même si quelqu'un connaît l'adresse de l'API et la clé publique de l'application, il ne peut pas lire ou modifier les mêmes d'un autre utilisateur. La protection opère au niveau du moteur PostgreSQL, pas au niveau du code applicatif.

### **Pourquoi Groq plutôt qu'OpenAI ou HuggingFace**

Groq propose une API gratuite sans carte bancaire, avec des temps de réponse inférieurs à deux secondes grâce à ses puces LPU dédiées à l'inférence. Le modèle Llama 4 Scout que j'utilise supporte l'analyse d'images, ce qui est indispensable pour analyser le contenu visuel d'un même et générer des légendes pertinentes.

### **Stabilité en production**

Le service Railway gratuit se met en veille après quinze minutes d'inactivité. Pour éviter que les utilisateurs attendent plusieurs secondes au premier appel à l'IA, j'ai connecté UptimeRobot, un service de monitoring gratuit qui envoie une requête de vérification toutes les cinq minutes à l'adresse de santé du backend. Ce mécanisme maintient le serveur actif en permanence.

## **Difficultés rencontrées et solutions**

Ce projet m'a confronté à plusieurs problèmes techniques concrets que je n'avais pas anticipés.

### **Le modèle IA a été supprimé en cours de développement**

Le modèle llama-3.2-11b-vision-preview que j'utilisais initialement a été décommissionné par Groq pendant le développement. J'ai dû identifier le nouveau modèle disponible, meta-llama/llama-4-scout-17b-16e-instruct, et adapter le code en conséquence. Cela m'a appris à ne pas coder en dur le nom d'un modèle dans le code source, mais à le passer en variable d'environnement pour pouvoir le changer sans redéploiement.

### **Les requêtes du backend étaient bloquées par Groq**

Sur le plan gratuit de Render, les serveurs partagent des adresses IP qui ont été blacklistées par Groq à cause d'abus. Toutes mes requêtes retournaient une erreur 401. J'ai migré vers Railway dont les adresses IP ne sont pas bloquées, ce qui a immédiatement résolu le problème.

### **Le canvas exportait une image vide lors du deuxième enregistrement**

Fabric.js avait un comportement inattendu : après un premier appel à la méthode toDataURL, le canvas semblait perdre ses références internes lors du deuxième appel si l'objet actif n'avait pas été désélectionné. La solution a été de forcer la désélection de tous les objets et de déclencher un nouveau rendu complet avant chaque export, en utilisant la méthode toCanvasElement pour créer une copie native indépendante du canvas Fabric.

### **La gestion des variables d'environnement sur Windows**

L'outil dotenvx installé globalement sur ma machine interceptait le fichier .env et injectait une ancienne version des variables, écrasant la nouvelle clé API. J'ai contourné le problème en lisant manuellement le fichier .env dans le code Node.js avec la librairie native fs, en forçant l'écrasement des variables existantes dans process.env.

## **Conclusion**

Ce projet m'a permis de mettre en pratique un ensemble de compétences que je souhaite approfondir dans le cadre du Master IA de SUPINFO : intégration d'API d'intelligence artificielle, sécurité des données utilisateurs, architecture d'une application full-stack, et déploiement en production.

Au-delà des exigences techniques du sujet, j'ai choisi d'aller plus loin en intégrant une authentification complète, une isolation des données par Row Level Security, et un modèle de vision multimodale réel. L'ensemble fonctionne en production sur une infrastructure entièrement gratuite.

Les problèmes rencontrés — modèle décommissionné, IP bloquées, comportements inattendus de Fabric.js, conflits de variables d'environnement — m'ont autant appris que le développement lui-même. Chaque blocage m'a forcé à comprendre en profondeur le système plutôt que de contourner le problème.

L'application est disponible à l'adresse <https://memegen-supinfo.vercel.app> et le code source documenté sur GitHub.